

Coral Shop

Plataforma de comercio electrónico para la venta de ropa juvenil personalizable

Avance 2 · Arquitectura de la solución: capas, cliente-servidor, MVC y eventos

CURSO	DOCENTE	AÑO
Desarrollo Web Integrado	Joel Ronald Vilca Chambi	2026
GRUPO 1		
Choquehuanca Marrufo, Liam Lennon · Damián Valdivia, Diego Aarón · Loayza Gerónimo, Juan Franco · Villanueva Montalvo, Apolo Chris · Campos Sulca, Jian Pier		

Lo que vamos a recorrer

1

Dónde estamos

Fases terminadas y el caso de los 12 polos

2

Arquitectura en capas

El restaurante y el recorrido de un pedido

3

Cliente-servidor

Varios clientes, un servidor, una base

4

MVC

React como vista, Spring como controlador

5

Dirigida por eventos

OrderStatusChanged, la propuesta de la fase 4

6

Implementación

Paquetes, reglas, pruebas y próximos pasos

Tres fases terminadas e integradas



25

tablas en PostgreSQL

13

módulos del backend

40

pruebas unitarias

64

controles de punta a punta

EL CASO QUE GUÍA TODO EL AVANCE

Un cliente personaliza 12 polos con su logo bordado en el pecho (4 S, 4 M, 4 L), crea el pedido, paga, y el administrador lo avanza de PAGADO a ENTREGADO. Total verificado: S/ 591.60 + S/ 10.00 de envío.

Cuatro patrones, un mismo sistema

1 · CAPAS

Organiza el servidor

Presentación → Negocio → Datos. Cada módulo tiene controller, service y repository.

2 · CLIENTE-SERVIDOR

Reparte qué corre dónde

React en el navegador, Spring Boot en el servidor y PostgreSQL detrás.

3 · MVC

Separa datos, pantalla y coordinación

Modelo: records y servicios. Vista: React. Controlador: @RestController.

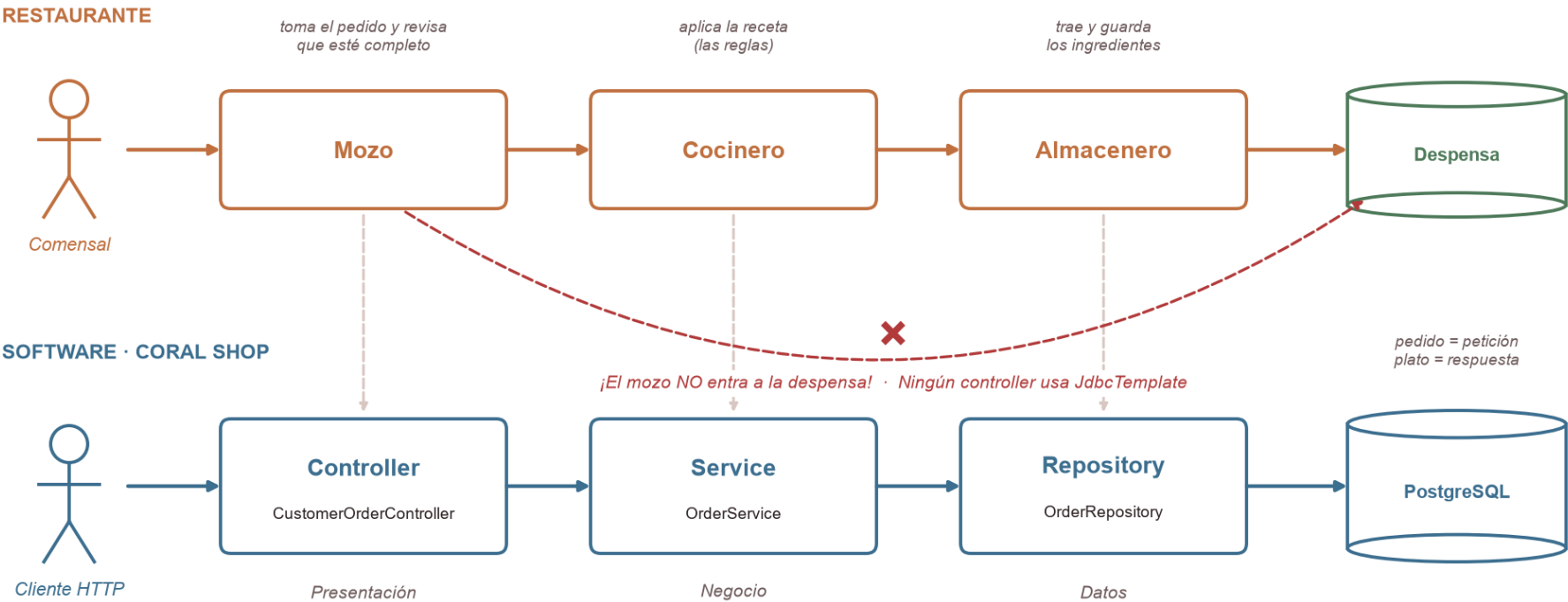
4 · DIRIGIDO POR EVENTOS

Reacciona a hechos del negocio

OrderStatusChanged publicado en el bus de Spring. Diseñado para la fase 4.

El mozo no entra a la despensa

FIGURA A1 · ARQUITECTURA EN CAPAS
El restaurante en capas aplicado a Coral Shop



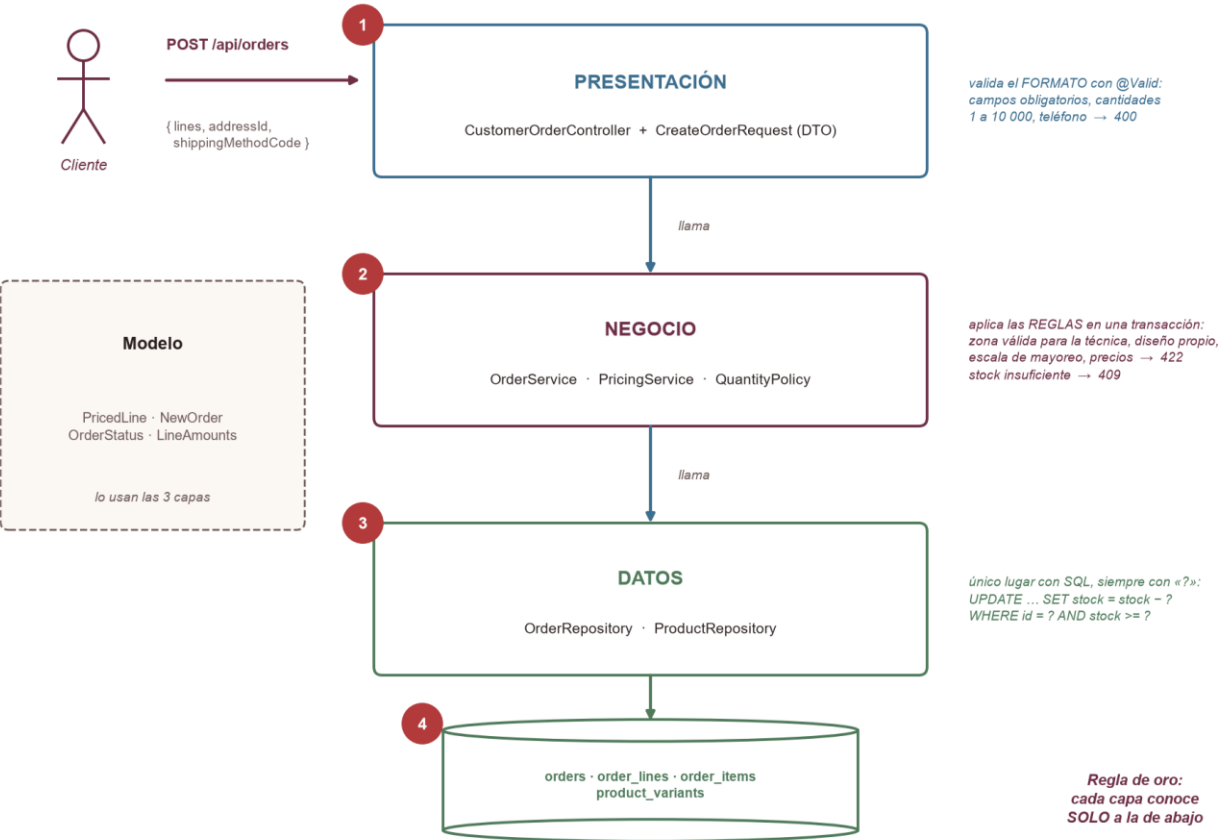
Cada capa solo conversa con la de al lado: si cambia la base de datos, el mozo y el cocinero no se enteran.

En el código: ningún controller recibe JdbcTemplate ni escribe SQL; todo el SQL vive en los @Repository.

Crear un pedido: tres capas, tres responsabilidades

FIGURA A2 · ARQUITECTURA EN CAPAS

Crear un pedido en Coral Shop: el recorrido por las tres capas



- **Presentación.** El DTO valida el formato con `@Valid`; si falla, 400 sin tocar la base.
- **Negocio.** Una transacción: producto activo, zona válida para la técnica, diseño propio, escala de mayoreo y precios recalculados.
- **Datos.** Descuento condicional de stock; si no alcanza, rollback completo y 409.
- **Modelo compartido.** `PricedLine`, `NewOrder` y `OrderStatus` los usan las tres capas.

Cada capa responde con su propio código

400

Presentación

Formato inválido

Cotización sin líneas · archivo que no es imagen · diseño de más de 5 MB

404

Negocio

No existe para ti

Un cliente pide el detalle de un pedido ajeno

409

Negocio · Datos

Conflicto

«Stock insuficiente para POLO-...-S (S, Blanco): pediste 21»

422

Negocio

Regla violada

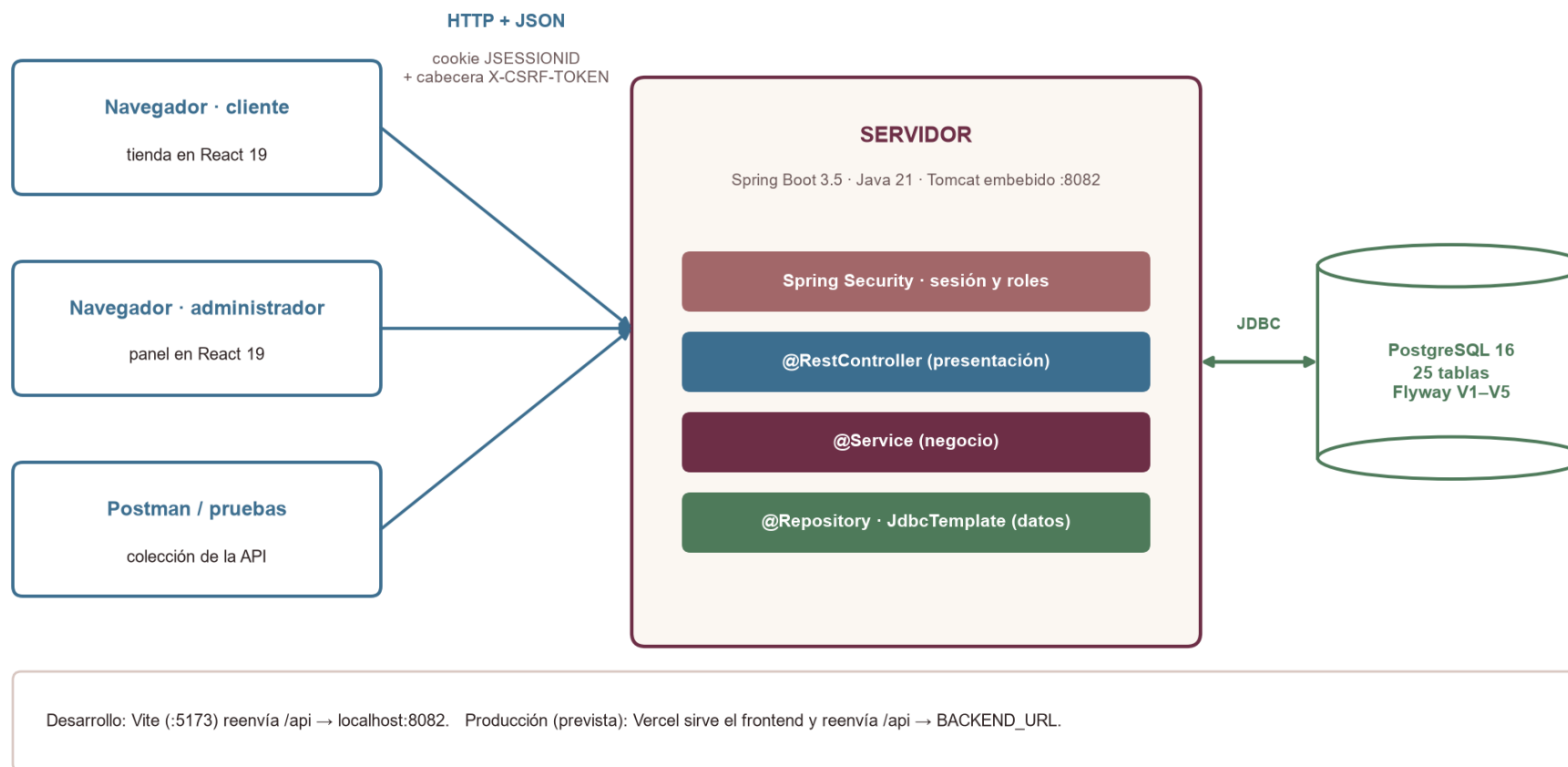
«La zona ESPALDA no admite bordado...» · pasar de PAGADO a ENTREGADO

Un único `@RestControllerAdvice` devuelve siempre `{ status, message, errors }`: mensajes reales obtenidos al verificar la API.

El servidor es la única autoridad

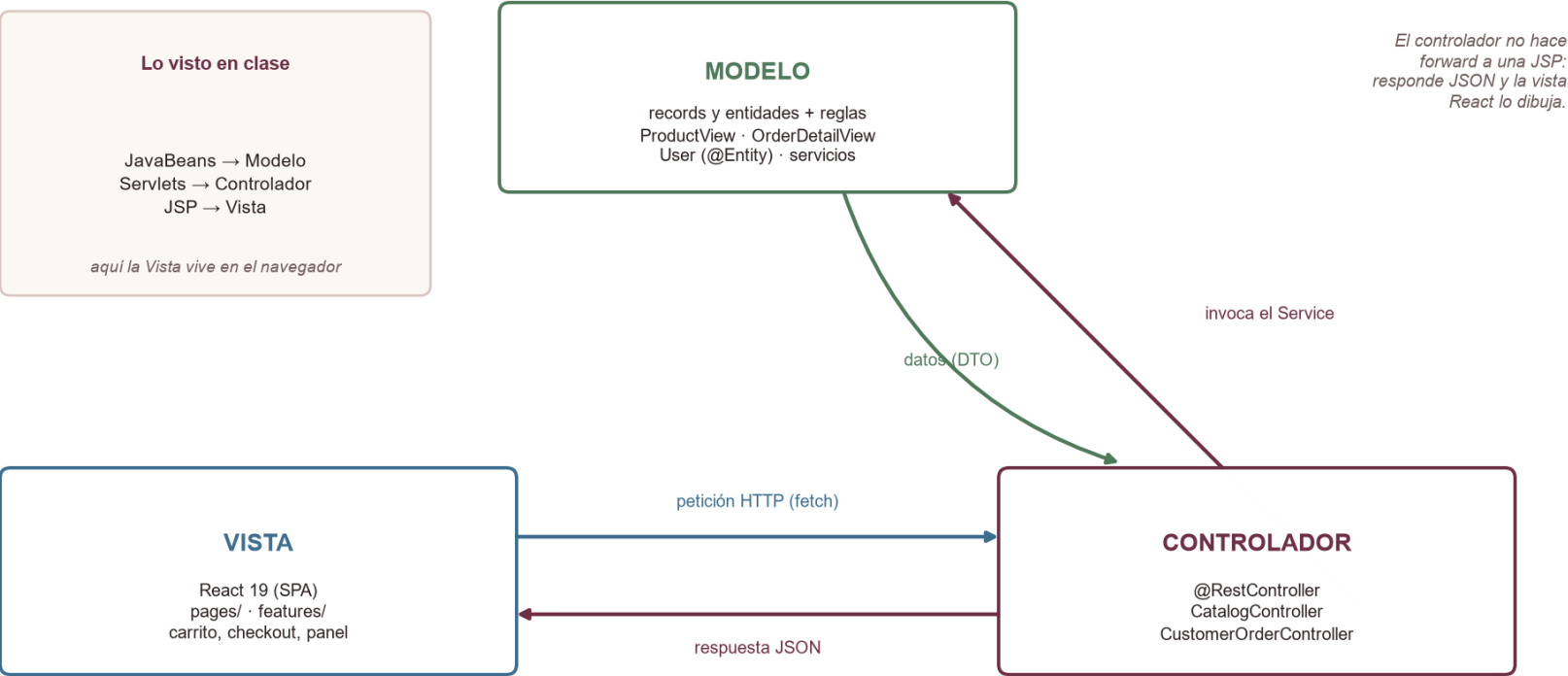
FIGURA A3 · ARQUITECTURA CLIENTE-SERVIDOR

Varios clientes, un servidor de aplicación y una base de datos



Los tres roles de clase, con la vista en el navegador

FIGURA A4 · MODELO-VISTA-CONTROLADOR
MVC con React como vista y Spring como controlador

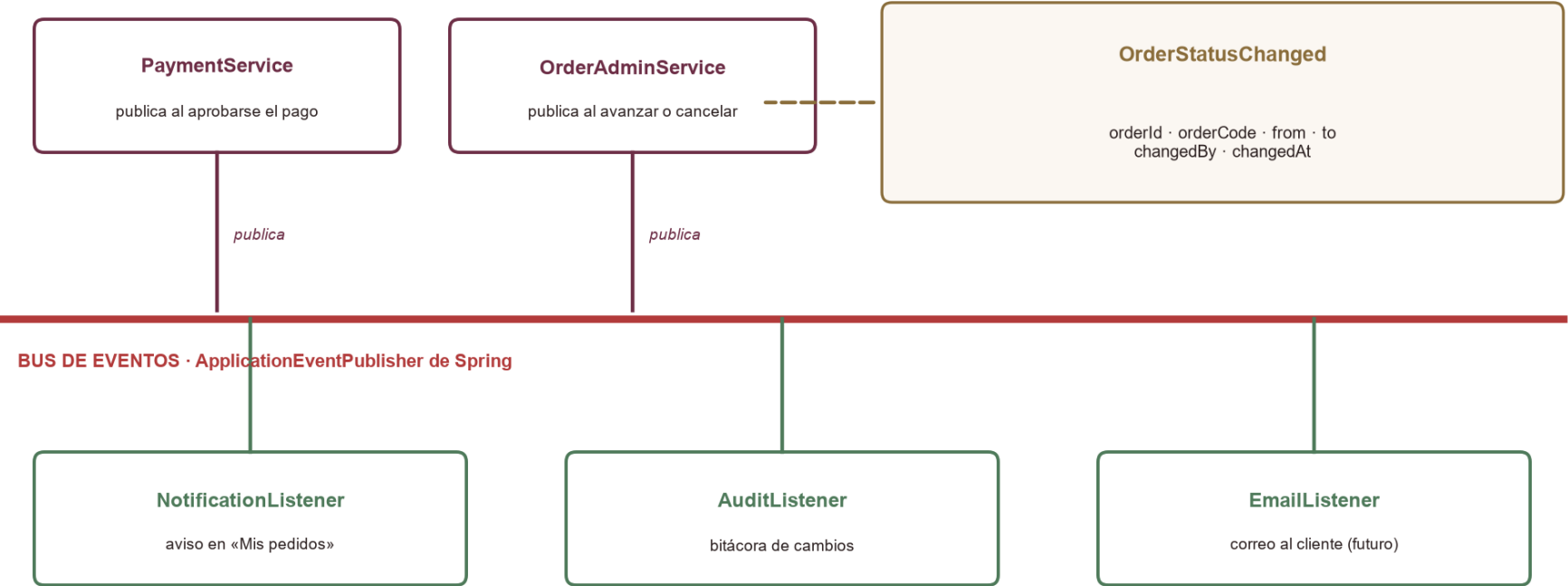


Quien publica no conoce a quien escucha

FIGURA A5 · ARQUITECTURA DIRIGIDA POR EVENTOS

Propuesta para la fase 4: el evento OrderStatusChanged

PROPUESTA · AÚN NO IMPLEMENTADO



@TransactionalEventListener(AFTER_COMMIT); los suscriptores reaccionan solo si el cambio de estado se guardó. Quien publica no conoce a quien escucha.

Cómo conviven los cuatro patrones

PATRÓN	NIVEL	BENEFICIO PARA CORAL SHOP
Cliente-servidor	Despliegue	Reglas y datos protegidos en un solo lugar
MVC	Interacción	La misma API sirve a la tienda, al panel y a las pruebas
Capas	Organización del servidor	Cambios acotados y errores 400, 422 y 409 bien separados
Eventos	Reacciones del negocio	Notificar y auditar sin tocar el flujo de pedidos

Paquetes por funcionalidad y por capa

```
com.coralshop/  
├─ auth/          config · controller · dto · service  
├─ catalog/       controller · dto · model · repository · service  
├─ customization/ controller · dto · model · repository · service  
├─ design/        controller · dto · model · repository · service  
├─ pricing/       controller · dto · model · service  
├─ order/         controller · dto · model · repository · service  
├─ payment/       controller · dto · model · repository · service  
├─ address/ · shipping/ · user/ · stats/ · health/  
└─ common/        dto · exception  
  
db/migration/     V1 ... V5 (Flyway)
```

Cliente

Catálogo filtrado, personalizador, diseños, cotización, direcciones, pedido, pago y «Mis pedidos».

Administrador

Bandeja de pedidos con máquina de estados, productos, categorías, usuarios y métricas.

Seguridad

Sesión, CSRF, roles (un admin también compra) y SQL siempre con parámetros.

Reglas del negocio, verificadas

- **Precio en el servidor:** base + técnica + zonas, con el descuento de la escala.
- **Mayoreo:** 1–2 sin descuento · 3–5 al 5 % · 6–11 al 10 % · 12+ al 15 %.
- **Zonas por tipo:** el bordado admite menos zonas y tamaños que el estampado.
- **Sin sobreventa:** UPDATE ... WHERE stock >= ? dentro de la transacción.
- **Estados:** PAGADO solo con pago aprobado; cancelar repone el stock.
- **Pago seguro:** pedido bloqueado durante el cobro; la tarjeta no se guarda.

40 / 40

pruebas unitarias

QuantityPolicy, importes, estados, imagen y pasarela

64 / 64

controles de punta a punta

flujo completo y cada regla rota contra PostgreSQL

0

errores 500

en el registro del servidor durante la verificación

Del API a la tienda completa

Fase 3

Frontend de compra

Personalizador con vista previa, carrito por diseño, checkout, pago y «Mis pedidos».

Fase 4

Evento de dominio

OrderStatusChanged con notificación al cliente y auditoría.

Fase 5

Calidad y demo

Datos ficticios, colección Postman, informe y presentación finales.

Despliegue

Vercel + backend

Frontend en Vercel con la variable `BACKEND_URL`; backend en el servicio que se elija.

PARA CERRAR

La arquitectura se ve en el código

Capas comprobables

Ningún controller toca la base; cada capa responde con su código.

Servidor como autoridad

Precios y stock se recalculan siempre en el servidor.

MVC adaptado

La vista pasó de JSP a React; el controlador responde JSON.

Eventos con propósito

Se agregan en la fase 4 para notificar y auditar.

Gracias. ¿Preguntas?